

# 系统架构设计说明书

校园二手物品交易平台

项目组

版本：V1.0

日期：2026-07-04

# 目录

|          |                |          |
|----------|----------------|----------|
| <b>1</b> | <b>引言</b>      | <b>3</b> |
| 1.1      | 编写目的           | 3        |
| 1.2      | 适用范围           | 3        |
| 1.3      | 参考文档           | 3        |
| <b>2</b> | <b>系统总体架构</b>  | <b>3</b> |
| 2.1      | 架构风格           | 3        |
| 2.2      | 技术选型           | 3        |
| 2.3      | 系统架构层次         | 4        |
| <b>3</b> | <b>前端架构设计</b>  | <b>4</b> |
| 3.1      | 项目目录结构         | 4        |
| 3.2      | 路由设计           | 5        |
| 3.3      | 状态管理设计 (Pinia) | 6        |
| 3.4      | 前端数据流          | 6        |
| <b>4</b> | <b>后端架构设计</b>  | <b>6</b> |
| 4.1      | 项目目录结构         | 6        |
| 4.2      | 分层架构           | 7        |
| 4.3      | 统一响应格式         | 7        |
| <b>5</b> | <b>数据库设计</b>   | <b>8</b> |
| 5.1      | 实体关系           | 8        |
| 5.2      | 数据库表结构         | 8        |
| 5.2.1    | 用户表 (user)     | 8        |
| 5.2.2    | 商品表 (product)  | 8        |
| 5.2.3    | 订单表 (order)    | 9        |
| 5.3      | Redis 缓存策略     | 9        |
| <b>6</b> | <b>接口设计</b>    | <b>9</b> |
| 6.1      | 接口规范           | 9        |
| 6.2      | 用户模块接口         | 10       |
| 6.3      | 商品模块接口         | 10       |
| 6.4      | 订单模块接口         | 10       |
| 6.5      | 后台管理接口         | 11       |

|          |                     |           |
|----------|---------------------|-----------|
| <b>7</b> | <b>安全架构设计</b>       | <b>11</b> |
| 7.1      | 认证与授权流程 . . . . .   | 11        |
| 7.2      | 安全防护措施 . . . . .    | 11        |
| <b>8</b> | <b>部署架构</b>         | <b>12</b> |
| 8.1      | 部署拓扑 . . . . .      | 12        |
| 8.2      | 环境要求 . . . . .      | 12        |
| <b>9</b> | <b>关键技术方案</b>       | <b>12</b> |
| 9.1      | 事务管理 . . . . .      | 12        |
| 9.2      | 并发控制 . . . . .      | 13        |
| 9.3      | 文件上传方案 . . . . .    | 13        |
| 9.4      | 日志方案 . . . . .      | 13        |
| <b>A</b> | <b>架构决策记录 (ADR)</b> | <b>14</b> |
| <b>B</b> | <b>版本记录</b>         | <b>14</b> |

# 1 引言

## 1.1 编写目的

本文档旨在对校园二手物品交易平台进行全面的系统架构设计，明确系统的技术选型、架构风格、模块划分、数据库设计、接口规范及部署方案，为后续的详细设计与编码实现提供技术指导。

## 1.2 适用范围

本文档适用于本项目的系统架构师、后端开发工程师、前端开发工程师、测试工程师及项目管理人员。

## 1.3 参考文档

| 序号 | 文档名称        |
|----|-------------|
| 1  | 《软件需求规格说明书》 |
| 2  | 《项目指导书》     |

# 2 系统总体架构

## 2.1 架构风格

本系统采用**前后端分离架构**，前端为单页应用（SPA），后端提供 RESTful API 服务。前后端通过 HTTP/HTTPS 协议进行数据交互，使用 JWT 实现无状态认证。

## 2.2 技术选型

| 层级       | 技术           | 版本   | 说明                   |
|----------|--------------|------|----------------------|
| 前端框架     | Vue 3        | 3.4+ | 组合式 API + TypeScript |
| UI 组件库   | Element Plus | 2.5+ | 企业级 UI 组件            |
| 状态管理     | Pinia        | 2.1+ | Vue3 官方状态管理          |
| 路由管理     | Vue Router   | 4.2+ | SPA 路由控制             |
| HTTP 客户端 | Axios        | 1.6+ | 请求拦截器、统一错误处理         |
| 后端框架     | Spring Boot  | 3.2+ | 微服务快速开发框架            |

| 层级     | 技术              | 版本    | 说明         |
|--------|-----------------|-------|------------|
| 安全框架   | Spring Security | 6.x+  | 认证与授权      |
| ORM 框架 | MyBatis-Plus    | 3.5+  | 数据库操作增强    |
| 数据库    | MySQL           | 8.0+  | 关系型数据库     |
| 缓存     | Redis           | 6.0+  | 高性能缓存      |
| 令牌     | JWT             | 0.12+ | 无状态认证      |
| 接口文档   | Knife4j         | 4.3+  | Swagger 增强 |
| 项目构建   | Maven           | 3.9+  | 后端构建工具     |
| 构建工具   | Vite            | 5.0+  | 前端构建工具     |

## 2.3 系统架构层次

系统自上而下划分为四层：

1. **客户端层：** Vue3 单页应用，Element Plus UI 组件，Axios HTTP 通信
2. **网关层：** SpringBoot 统一入口，CORS 跨域配置，JWT 认证过滤器
3. **业务层：** 用户模块、商品模块、订单模块、消息模块、后台管理模块
4. **数据层：** MySQL 关系型数据库、Redis 缓存

## 3 前端架构设计

### 3.1 项目目录结构

```
1  zhuanzhuan-frontend/
2      public/                # 静态资源
3      src/
4          api/                # API接口层
5              request.ts      # Axios 实例与拦截器
6              user.ts         # 用户模块接口
7              product.ts      # 商品模块接口
8              order.ts        # 订单模块接口
9              message.ts      # 消息模块接口
10     assets/                 # 静态资源
11     components/             # 公共组件
12         common/             # 通用组件
13         product/            # 商品相关组件
```

```

14         layout/           # 布局组件
15     composables/         # 组合式函数
16     layouts/             # 布局模板
17     router/              # 路由配置
18     stores/              # Pinia 状态管理
19         user.ts          # 用户状态
20         app.ts           # 应用状态
21     utils/               # 工具函数
22     views/               # 页面组件
23         home/            # 首页
24         product/         # 商品相关页面
25         order/           # 订单相关页面
26         user/            # 个人中心页面
27         message/        # 消息页面
28         admin/           # 后台管理页面
29     App.vue
30     main.ts
31 index.html
32 vite.config.ts
33 tsconfig.json
34 package.json
    
```

### 3.2 路由设计

| 路由路径             | 页面组件               | 权限 | 说明   |
|------------------|--------------------|----|------|
| /                | HomePage.vue       | 公开 | 首页   |
| /login           | LoginPage.vue      | 公开 | 登录页  |
| /register        | RegisterPage.vue   | 公开 | 注册页  |
| /product/list    | ProductList.vue    | 公开 | 商品列表 |
| /product/:id     | ProductDetail.vue  | 公开 | 商品详情 |
| /product/publish | ProductPublish.vue | 卖家 | 发布商品 |
| /cart            | CartPage.vue       | 用户 | 购物车  |
| /order/list      | OrderList.vue      | 用户 | 订单列表 |
| /order/:id       | OrderDetail.vue    | 用户 | 订单详情 |
| /user/profile    | UserProfile.vue    | 用户 | 个人中心 |
| /user/address    | AddressManage.vue  | 用户 | 地址管理 |
| /user/favorites  | UserFavorites.vue  | 用户 | 我的收藏 |

| 路由路径                 | 页面组件                   | 权限  | 说明   |
|----------------------|------------------------|-----|------|
| /message             | MessagePage.vue        | 用户  | 站内信  |
| /admin/users         | AdminUsers.vue         | 管理员 | 用户管理 |
| /admin/products      | AdminProducts.vue      | 管理员 | 商品审核 |
| /admin/orders        | AdminOrders.vue        | 管理员 | 订单管理 |
| /admin/stats         | AdminStats.vue         | 管理员 | 数据统计 |
| /admin/announcements | AdminAnnouncements.vue | 管理员 | 公告管理 |

### 3.3 状态管理设计 (Pinia)

前端使用 Pinia 进行状态管理，主要 Store 包括：

- **userStore**: 用户信息、JWT Token、登录状态
- **appStore**: 系统配置、主题设置、全局 Loading
- **productStore**: 商品列表缓存、分类缓存、搜索条件
- **orderStore**: 当前订单、订单筛选条件
- **messageStore**: 未读消息数、会话列表

### 3.4 前端数据流

数据流向为：用户操作 → Vue Component → Pinia Action → API Call (Axios) → SpringBoot Backend → API Response → Pinia State → Vue Component → 用户界面。

## 4 后端架构设计

### 4.1 项目目录结构

```

1 zhuanzhuan-backend/
2   src/main/java/com/zhuanzhuan/
3     ZhuanZhuanApplication.java      # 启动类
4   common/                           # 公共模块
5     config/                         # 配置类
6     constant/                       # 常量定义
7     exception/                     # 异常处理
8     result/                         # 统一返回结果

```

```

9         util/                # 工具类
10        modules/             # 业务模块
11        user/                 # 用户模块
12        product/             # 商品模块
13        order/                # 订单模块
14        message/              # 消息模块
15        admin/                # 后台管理模块
16        security/             # 安全模块
17    src/main/resources/
18        application.yml
19        application-dev.yml
20        application-prod.yml
21    pom.xml
22    README.md

```

## 4.2 分层架构

后端采用经典的四层架构：

| 层次           | 职责                                       |
|--------------|------------------------------------------|
| Controller 层 | 接收 HTTP 请求、参数校验、调用 Service、返回 RESTful 响应 |
| Service 层    | 核心业务逻辑、事务管理、业务规则校验                       |
| Mapper 层     | MyBatis-Plus 数据库 CRUD、复杂查询               |
| Entity 层     | POJO 实体类、DTO 数据传输对象、VO 视图对象              |

## 4.3 统一响应格式

```

1 // 成功响应
2 {"code": 200, "message": "success", "data": {}}
3
4 // 失败响应
5 {"code": 400, "message": "参数错误", "data": null}
6
7 // 分页响应
8 {"code": 200, "message": "success",
9  "data": {"records": [], "total": 100, "page": 1, "size": 10}}

```



## 5 数据库设计

### 5.1 实体关系

系统包含以下核心实体：用户 (User)、商品 (Product)、商品图片 (ProductImage)、商品分类 (Category)、订单 (Order)、订单日志 (OrderLog)、消息 (Message)、通知 (Notification)、收藏 (Favorite)、收货地址 (Address)、交易评价 (Review)、公告 (Announcement)。

### 5.2 数据库表结构

#### 5.2.1 用户表 (user)

| 字段名           | 类型       | 长度  | 主键 | 非空 | 说明              |
|---------------|----------|-----|----|----|-----------------|
| id            | BIGINT   | 20  |    |    | 用户 ID           |
| username      | VARCHAR  | 50  |    |    | 用户名             |
| password_hash | VARCHAR  | 255 |    |    | 密码 (BCrypt)     |
| email         | VARCHAR  | 100 |    |    | 邮箱              |
| phone         | VARCHAR  | 20  |    |    | 手机号             |
| avatar        | VARCHAR  | 255 |    |    | 头像 URL          |
| role          | ENUM     |     |    |    | 角色 (user/admin) |
| credit_score  | INT      | 11  |    |    | 信誉分             |
| status        | TINYINT  | 4   |    |    | 状态 (0 禁用/1 启用)  |
| created_at    | DATETIME |     |    |    | 创建时间            |

#### 5.2.2 商品表 (product)

| 字段名         | 类型      | 长度   | 主键 | 非空 | 说明     |
|-------------|---------|------|----|----|--------|
| id          | BIGINT  | 20   |    |    | 商品 ID  |
| user_id     | BIGINT  | 20   |    |    | 卖家 ID  |
| category_id | BIGINT  | 20   |    |    | 分类 ID  |
| title       | VARCHAR | 100  |    |    | 商品标题   |
| description | TEXT    |      |    |    | 商品描述   |
| price       | DECIMAL | 10,2 |    |    | 价格 (元) |
| condition   | ENUM    |      |    |    | 成色     |
| status      | ENUM    |      |    |    | 状态     |

| 字段名        | 类型       | 长度 | 主键 | 非空 | 说明   |
|------------|----------|----|----|----|------|
| view_count | INT      | 11 |    |    | 浏览量  |
| created_at | DATETIME |    |    |    | 发布时间 |

### 5.2.3 订单表 (order)

| 字段名         | 类型       | 长度   | 主键 | 非空 | 说明       |
|-------------|----------|------|----|----|----------|
| id          | BIGINT   | 20   |    |    | 订单 ID    |
| order_no    | VARCHAR  | 32   |    |    | 订单号 (唯一) |
| buyer_id    | BIGINT   | 20   |    |    | 买家 ID    |
| seller_id   | BIGINT   | 20   |    |    | 卖家 ID    |
| product_id  | BIGINT   | 20   |    |    | 商品 ID    |
| total_price | DECIMAL  | 10,2 |    |    | 总价       |
| status      | ENUM     |      |    |    | 订单状态     |
| created_at  | DATETIME |      |    |    | 创建时间     |

## 5.3 Redis 缓存策略

| 缓存对象 | Key 设计                | 过期时间  | 说明           |
|------|-----------------------|-------|--------------|
| 验证码  | captcha:{email/phone} | 5 分钟  | 注册/找回密码      |
| 用户会话 | session:{userId}      | 7 天   | JWT Token 刷新 |
| 热点商品 | product:{id}          | 30 分钟 | 商品详情缓存       |
| 商品分类 | category:tree         | 24 小时 | 分类树结构        |
| 首页推荐 | index:recommend       | 10 分钟 | 按热度排序        |
| 接口防刷 | rate:limit:{ip}:{api} | 1 分钟  | 频率限制         |

## 6 接口设计

### 6.1 接口规范

- 接口风格： RESTful API
- 基础路径： /api/v1/
- 数据格式： JSON (UTF-8)

- **认证方式：** JWT Token (Authorization: Bearer)
- **分页参数：** page (页码)、size (每页条数)

## 6.2 用户模块接口

| 方法     | 路径                        | 说明     | 权限 |
|--------|---------------------------|--------|----|
| POST   | /api/v1/user/register     | 用户注册   | 公开 |
| POST   | /api/v1/user/login        | 用户登录   | 公开 |
| POST   | /api/v1/user/code         | 发送验证码  | 公开 |
| GET    | /api/v1/user/info         | 获取个人信息 | 登录 |
| PUT    | /api/v1/user/info         | 更新个人信息 | 登录 |
| POST   | /api/v1/user/avatar       | 上传头像   | 登录 |
| GET    | /api/v1/user/address      | 地址列表   | 登录 |
| POST   | /api/v1/user/address      | 新增地址   | 登录 |
| PUT    | /api/v1/user/address/{id} | 编辑地址   | 登录 |
| DELETE | /api/v1/user/address/{id} | 删除地址   | 登录 |

## 6.3 商品模块接口

| 方法     | 路径                       | 说明        | 权限 |
|--------|--------------------------|-----------|----|
| GET    | /api/v1/product/list     | 商品列表 (分页) | 公开 |
| GET    | /api/v1/product/{id}     | 商品详情      | 公开 |
| POST   | /api/v1/product          | 发布商品      | 卖家 |
| PUT    | /api/v1/product/{id}     | 编辑商品      | 卖家 |
| DELETE | /api/v1/product/{id}     | 删除商品      | 卖家 |
| POST   | /api/v1/product/favorite | 收藏/取消收藏   | 登录 |
| GET    | /api/v1/category/list    | 分类列表      | 公开 |

## 6.4 订单模块接口

| 方法   | 路径                 | 说明   | 权限 |
|------|--------------------|------|----|
| POST | /api/v1/order      | 创建订单 | 登录 |
| GET  | /api/v1/order/list | 订单列表 | 登录 |
| GET  | /api/v1/order/{id} | 订单详情 | 登录 |

| 方法   | 路径                         | 说明   | 权限 |
|------|----------------------------|------|----|
| PUT  | /api/v1/order/{id}/pay     | 支付订单 | 登录 |
| PUT  | /api/v1/order/{id}/ship    | 确认发货 | 卖家 |
| PUT  | /api/v1/order/{id}/receive | 确认收货 | 买家 |
| POST | /api/v1/order/{id}/review  | 评价订单 | 登录 |

## 6.5 后台管理接口

| 方法   | 路径                                | 说明      | 权限  |
|------|-----------------------------------|---------|-----|
| GET  | /api/v1/admin/users               | 用户列表    | 管理员 |
| PUT  | /api/v1/admin/user/{id}/status    | 禁用/启用用户 | 管理员 |
| GET  | /api/v1/admin/product/review      | 待审核商品   | 管理员 |
| PUT  | /api/v1/admin/product/review/{id} | 审核商品    | 管理员 |
| GET  | /api/v1/admin/orders              | 订单管理    | 管理员 |
| GET  | /api/v1/admin/statistics          | 数据统计    | 管理员 |
| POST | /api/v1/admin/announcement        | 发布公告    | 管理员 |

## 7 安全架构设计

### 7.1 认证与授权流程

用户认证基于 JWT Token 实现，授权基于 SpringSecurity 的 RBAC 模型：

1. 用户登录后服务端生成 JWT Token（包含用户 ID、角色等信息）
2. 客户端将 Token 存储于 localStorage，后续请求通过 Authorization 头携带
3. 后端 JWT 认证过滤器解析 Token，校验合法性后设置 SecurityContext
4. SpringSecurity 根据用户角色进行接口级权限控制

### 7.2 安全防护措施

| 防护类型     | 措施                                    |
|----------|---------------------------------------|
| XSS 防护   | 前端对用户输入做转义处理；后端使用 XSS 过滤库             |
| CSRF 防护  | SameSite=Lax Cookie 策略；关键接口校验 Referer |
| SQL 注入防护 | MyBatis-Plus 参数绑定；禁止字符串拼接 SQL         |

| 防护类型 | 措施                    |
|------|-----------------------|
| 接口防刷 | Redis 实现接口频率限制（IP 级别） |
| 数据脱敏 | 手机号中间 4 位隐藏；邮箱部分隐藏    |
| 密码安全 | BCrypt 加密存储，随机盐值      |

## 8 部署架构

### 8.1 部署拓扑

系统部署分为四层：

1. **负载均衡层：** Nginx 反向代理，SSL 终结，静态资源托管
2. **前端层：** Vue3 SPA 构建产物由 Nginx 托管
3. **后端层：** SpringBoot Jar 多实例部署，提供 RESTful API
4. **数据层：** MySQL 主从复制 + Redis 哨兵模式

### 8.2 环境要求

| 环境     | 配置                              | 数量 |
|--------|---------------------------------|----|
| 前端服务器  | 2 核 4GB + Ubuntu 20.04          | 1  |
| 后端服务器  | 4 核 8GB + JDK 17 + Ubuntu 20.04 | 2  |
| 数据库服务器 | 4 核 8GB + MySQL 8.0             | 1  |
| 缓存服务器  | 2 核 4GB + Redis 6.0             | 1  |

## 9 关键技术方案

### 9.1 事务管理

| 场景   | 策略             | 说明                 |
|------|----------------|--------------------|
| 订单创建 | @Transactional | 订单创建 + 库存扣减原子性     |
| 支付回调 | @Transactional | 支付状态更新 + 商品状态更新原子性 |
| 用户注册 | @Transactional | 用户创建 + 默认地址原子性     |

## 9.2 并发控制

- **乐观锁**：商品表使用 version 字段，更新时校验版本号
- **Redis 分布式锁**：订单创建时使用 SETNX 防止重复下单
- **数据库锁**：库存扣减使用 SELECT ... FOR UPDATE 行级锁

## 9.3 文件上传方案

- 开发/测试阶段：本地文件存储
- 生产阶段：云 OSS（推荐阿里云 OSS/七牛云）
- 图片处理：Thumbnailator 生成缩略图

## 9.4 日志方案

- 应用日志：SLF4J + Logback，分级输出，按日滚动归档
- 操作日志：AOP + 自定义注解，记录关键操作
- SQL 日志：开发环境开启，生产环境关闭

## A 架构决策记录 (ADR)

| 编号      | 决策                 | 备选方案           | 选择理由               |
|---------|--------------------|----------------|--------------------|
| ADR-001 | 前后端分离架构            | 单体架构           | 更好的解耦与团队并行开发       |
| ADR-002 | JWT 无状态认证          | Session 认证     | 无需服务端存储会话          |
| ADR-003 | MyBatis-Plus ORM   | JPA/Hibernate  | 灵活 SQL 控制，学习成本低    |
| ADR-004 | Element Plus UI    | Ant Design Vue | Vue3 生态最成熟的中文 UI 库 |
| ADR-005 | Vue Router History | Hash 模式        | URL 更美观            |

## B 版本记录

| 版本   | 日期         | 修改内容 | 修改人 |
|------|------------|------|-----|
| V1.0 | 2026-07-04 | 初始版本 | 项目组 |